Exploratory Data Analysis (EDA)

Step 1: Install & Import Required Libraries

Use Pandas for data handling, Seaborn/Matplotlib for visualization,

And spaCy for NLP tasks (tokenization, stopwords, lemmatization).

```
# Import the necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from wordcloud import WordCloud
import spacy
from collections import Counter
from sklearn.feature_extraction.text import CountVectorizer
import os
```

Step 2: Load Dataset

Load the dataset from a CSV file and inspect its structure.

```
# Load spaCy model (ensure you've installed it via: `python -m spacy download
en_core_web_sm`)
nlp = spacy.load("en_core_web_sm")

# Set plot style
sns.set_style("whitegrid")

# Load dataset
df = pd.read_csv("/Users/ndubuisiobioha/Desktop/Documents/Dissertation/Dissertation
Code/cybersecurity_news.csv")

# Display first few rows
df.head()
```

Step 3: Basic Data Overview

Check the dataset's structure, missing values, and duplicate entries.

```
# Check dataset info
df.info()

# Check for missing values
df.isnull().sum()

# Check for duplicate rows
df.duplicated().sum()
```

Step 4: Data Cleaning with spaCy

- Convert text to lowercase
- Remove punctuation, extra space, and numbers
- Tokenize text and remove stopwords
- Apply lemmatization (convert words to their base form)

```
# Function to clean text using spaCy
def preprocess_text(text):
    doc = nlp(text.lower().strip())  # Convert to lowercase
    tokens = [token.lemma_ for token in doc if token.is_alpha and not token.is_stop]  #
Lemmatization & stopword removal
    return " ".join(tokens)

# Apply text preprocessing to the 'Article' column
df["cleaned_article"] = df["Article"].astype(str).apply(preprocess_text)

# Display first 20 rows of the cleaned data
df.head(20)
```

Step 5: Threat Category Distribution

Visualize the number of articles per threat category using a bar plot

```
plt.figure(figsize=(8,5))
sns.countplot(x=df["Label"], palette="viridis", order=df["Label"].value_counts().index)
plt.title("Distribution of Cybersecurity Threat Categories")
plt.xlabel("Threat Category")
plt.ylabel("Count")
plt.xticks(rotation=45)
plt.show()
```

Step 6: Generate Word Cloud for Each Category

Create word clouds to visualize frequently occurring words in each cybersecurity threat category.

```
def plot_wordcloud(category):
    text = " ".join(df[df["Label"] == category]["cleaned_article"])
    wordcloud = WordCloud(width=800, height=400, background_color="black").generate(text)

    plt.figure(figsize=(10,5))
    plt.imshow(wordcloud, interpolation="bilinear")
    plt.axis("off")
    plt.title(f"Word Cloud - {category}", fontsize=14)
    plt.show()

# Generate word clouds for each category
for label in df["Label"].unique():
      plot_wordcloud(label)
```

Step 7: Most Common Words per Category

Extract the top 10 most frequently used words in each category.

```
def get_most_common_words(category, n=10):
    def get_most_common_words(category, n=10):
    words = " ".join(df[df["Label"] == category]["cleaned_article"]).split()
```

```
    word_freq = Counter(words)
    return word_freq.most_common(n)

for label in df["Label"].unique():
    print(f"\nMost common words in {label}:")
    print(get_most_common_words(label))
```

Step 8: Bigrams & Trigrams Analysis

Extract the most common word pairs (bigrams) and word triplets (trigrams).

```
def get_top_ngrams(corpus, ngram_range=(2,2), n=10):
    vec = CountVectorizer(ngram_range=ngram_range, stop_words="english").fit(corpus)
    bag_of_words = vec.transform(corpus)
    sum_words = bag_of_words.sum(axis=0)
    words_freq = [(word, sum_words[0, idx]) for word, idx in vec.vocabulary_.items()]
    words_freq = sorted(words_freq, key = lambda x: x[1], reverse=True)
    return words_freq[:n]

bigrams = get_top_ngrams(df["cleaned_article"], (2,2), 10)
trigrams = get_top_ngrams(df["cleaned_article"], (3,3), 10)

print("\nTop Bigrams:\n", bigrams)
print("\nTop Trigrams:\n", trigrams)
```

Data Preprocessing

Preprocessing Cybersecurity Threat News Dataset

This notebook covers the processing steps for our cybersecurity news dataset. Start:

Load the dataset

Clean and normalize text

Tokenize text using RoBERTa

Encode labels

Split data into training & testing sets

Save the processed dataset for model training

Visualize label distribution

Step 1: Import Required Libraries

Start by importing the necessary libraries for data preprocessing

```
# Basic Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# NLP Libraries
import re
import torch
from transformers import RobertaTokenizer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

# Ignore Warnings
import warnings
warnings.filterwarnings("ignore")

# Load RoBERTa Tokenizer
tokenizer = RobertaTokenizer.from_pretrained("roberta-base")
```

Step 2: Load the Dataset

Load the dataset and display the first few rows to understand its structure.

```
# Load the dataset
file_path = "/Users/ndubuisiobioha/Desktop/Documents/Dissertation/Dissertation
Code/cybersecurity_news.csv"  # Change this to your actual file path
```

```
df = pd.read_csv(file_path)

# Display first 20 rows
df.head(20)
```

Step 3: Data Cleaning & Normalization

Text data may contain unwanted elements like URLs, special characters, and extra spaces, Clean and normalize the text.

```
def clean_text(text):
    """
    Function to clean and normalize text data.
    Steps:
    - Remove URLs
    - Remove special characters
    - Convert text to lowercase
    - Remove extra spaces
    """
    text = re.sub(r"http\S+|www\S+|https\S+", '', text, flags=re.MULTILINE)  # Remove URLs
    text = re.sub(r'\W', ' ', text)  # Remove special characters
    text = re.sub(r'\s+', ' ', text).strip()  # Remove extra spaces
    text = text.lower()  # Convert to lowercase
    return text

# Apply cleaning to 'Article' column
df['cleaned_text'] = df['Article'].astype(str).apply(clean_text)

# Display first 20 rows of the cleaned text
df[['Article', 'cleaned_text']].head(20)
```

Step 4: Tokenization Using RoBERTa

Since RoBERTa is been used, tokenize text to prepare it for the model.

```
# Tokenize and pad sequences
MAX_LEN = 512  # RoBERTa max token length

def tokenize_text(text):
    return tokenizer(text, truncation=True, padding="max_length", max_length=MAX_LEN,
return_tensors="pt")

df['tokens'] = df['cleaned_text'].apply(tokenize_text)

# Show tokenized data
df['tokens'].head()
```

Step 5: Label Encoding

The model needs numerical labels. Convert threat categories into numerical values.

```
# Encode labels
label_encoder = LabelEncoder()
df['label_encoded'] = label_encoder.fit_transform(df['Label'])

# Show label mapping
label_mapping = dict(zip(label_encoder.classes_,
label_encoder.transform(label_encoder.classes_)))
print("Label Mapping:", label_mapping)

# Show encoded labels
df[['Label', 'label_encoded']].head(20)
```

Step 6: Save Processed Data as a Single File

Save the entire processed dataset.

```
# Convert tokenized data to tensors
df['input_ids'] = df['tokens'].apply(lambda x: x['input_ids'].squeeze().tolist())
```

```
# Select relevant columns for saving
processed_df = df[['cleaned_text', 'input_ids', 'label_encoded']]

# Save as a single file
processed_df.to_csv("/Users/ndubuisiobioha/Desktop/Documents/Dissertation/Dissertation
Code/preprocessed_cybersecurity_news.csv", index=False)

print("Processed dataset saved successfully as a single file!")
```

Step 7: Data Visualization

Visualize the distribution of that threat categories in the dataset.

```
plt.figure(figsize=(8, 5))
sns.countplot(x=df['Label'], palette='viridis')
plt.xticks(rotation=45)
plt.title("Cybersecurity Threat Categories Distribution")
plt.xlabel("Threat Type")
plt.ylabel("Count")
plt.show()
```

Decision Tree Model Training

1. Importing Required Libraries

This section loads the core libraries needed for data handling, NLP, machine learning, and visualization.

pandas, numpy → For dataset handling and numerical operations.

spaCy → Tokenization and word embeddings (`en_core_web_md`).

Counter → Vocabulary building.

scikit-learn → Training and evaluating the Decision Tree classifier.

matplotlib, seaborn → Visualization of evaluation metrics.

pickle → Saving trained models and vocabulary.

sys, warnings → Environment checks and warning suppression.

```
# Import the necessary libraries
import pandas as pd
```

```
import numpy as np
import spacy
from collections import Counter
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
%matplotlib inline

import matplotlib.pyplot as plt
import seaborn as sns
import pickle
import sys
import warnings
```

2. Environment Check

Warnings was suppressed for cleaner output and print the Python/NumPy versions to ensure reproducibility.

This step ensures the runtime environment is consistent with the project requirements.

```
warnings.filterwarnings("ignore")

print("Python Version:", sys.version)
print("NumPy Version:", np.__version__)
```

4. Load Dataset

The preprocessed cybersecurity news dataset was imported.

If the file path is incorrect or missing, the script exits gracefully with an error message.

This ensures portability and prevents execution failures on other systems.

```
file_path = "/Users/ndubuisiobioha/Desktop/Documents/Dissertation/Dissertation
Code/Preprocessed/preprocessed_cybersecurity_news.csv"
try:
    df = pd.read_csv(file_path)
except FileNotFoundError:
```

```
    print(f"ERROR: File not found at {file_path}")
    sys.exit(1)
```

5. Tokenization with spaCy

Custom function was defined to tokenize text:

Converts text to lowercase.

Removes punctuation and extra spaces.

Splits text into individual tokens.

The resulting tokenized sequences were stored in a new DataFrame column `tokens`.

```
def tokenize_text(text):
    doc = nlp(str(text).lower(), disable=["parser", "ner"])
    return [token.text for token in doc if not token.is_punct and not token.is_space]

df['tokens'] = df['cleaned_text'].apply(tokenize_text)
```

6. Vocabulary Construction

Vocabulary based on token frequency was built:

Collect all tokens across the dataset.

Retain the top 20,000 most frequent tokens.

Assign a unique integer index to each token.

Reserve index `0` for the `<PAD>` token, used for sequence padding.

```
max_vocab_size = 20000
all_tokens = [token for tokens in df['tokens'] for token in tokens]
vocab = Counter(all_tokens)
vocab = {word: idx + 1 for idx, (word, _) in enumerate(vocab.most_common(max_vocab_size))}
vocab['<PAD>'] = 0
```

7. Token-to-Index Conversion

Each tokenized sentence was converted into a sequence of indices:

Maps tokens to integers using the vocabulary.

Truncates sequences longer than `512` tokens.
Unknown words are replaced with `0`.

This prepares text data for embedding lookup.

```
max_len = 512
def tokens_to_indices(tokens, vocab, max_len=max_len):
    indices = [vocab.get(token, 0) for token in tokens][:max_len]
    return indices

df['indices'] = df['tokens'].apply(lambda x: tokens_to_indices(x, vocab))
```

8. Custom Padding Function
Since ML models require inputs of equal length, then it pad sequences:
All sequences are fixed to a length of `512`.
Padding is added at the end (`post-padding`).
The `<PAD>` token (`0`) is used for padding.

This ensures uniform input dimensions across the dataset.

```
def custom_pad_sequences(sequences, maxlen, padding='post', value=0):
    padded = np.full((len(sequences), maxlen), value, dtype=np.int32)
    for i, seq in enumerate(sequences):
        seq_len = min(len(seq), maxlen)
        if padding == 'post':
            padded[i, :seq_len] = seq[:seq_len]
        else:
            padded[i, -seq_len:] = seq[:seq_len]
    return padded

sequences = custom_pad_sequences(df['indices'].tolist(), maxlen=max_len, padding='post',
value=0)
labels = df['label_encoded'].values
```

9. Label Validation and Remapping

Distribution of class labels was checked:

Prints label counts to identify potential imbalance.

Remaps labels to a continuous integer range `[0, n_classes-1]`.

This is required since ML models expect labels in numeric format.

```
print("Label value counts:", pd.Series(labels).value_counts())
print("Min label:", labels.min(), "Max label:", labels.max())
label_map = {label: idx for idx, label in enumerate(sorted(np.unique(labels)))}
labels = np.array([label_map[label] for label in labels], dtype=np.int32)
print("Remapped label value counts:", pd.Series(labels).value_counts())
```

10. Embedding Matrix with spaCy

An embedding matrix was created where:

- Each row corresponds to a word vector from spaCy.
- Embedding dimension = `300`.
- Invalid or missing embeddings are replaced with zeros.

This matrix allows us to map tokens to dense numerical representations.

```
def create_embedding_matrix(vocab, nlp, embedding_dim=300):
    embedding_matrix = np.zeros((len(vocab) + 1, embedding_dim))
    for i, word in enumerate(vocab, 1):
        embedding_matrix[i] = nlp(word).vector
    return embedding_matrix

embedding_matrix = create_embedding_matrix(list(vocab.keys()), nlp, embedding_dim=300)
embedding_matrix = np.nan_to_num(embedding_matrix, nan=0.0, posinf=0.0, neginf=0.0)
```

11. Sequence to Document Embeddings

Each document is represented as the mean of its word embeddings:

This produces a fixed-size (300-dimensional) representation for each news article.

```
def sequence_to_embedding(sequence, embedding_matrix, max_len=512):
    valid_indices = sequence[sequence != 0]
    if len(valid_indices) == 0:
        return np.zeros(embedding_matrix.shape[1])
    embeddings = embedding_matrix[valid_indices]
    return np.mean(embeddings, axis=0)

X = np.array([sequence_to_embedding(seq, embedding_matrix, max_len) for seq in
sequences])
y = labels
```

12. Dataset Splitting

The dataset got splitted into:

Training set (80%)

Validation set (20%)

This ensures the model is trained on one portion of the data and evaluated on unseen data.

```
from sklearn.model_selection import train_test_split
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42)
```

13. Train Decision Tree Classifier

it is then initialize and train a Decision Tree Classifier:

`max_depth=10` → Limits tree depth to avoid overfitting.

`min_samples_split=5` → Ensures splits happen only with enough samples.

`random_state=42` → Guarantees reproducibility.

The classifier is trained on the averaged document embeddings.

```
dt_model = DecisionTreeClassifier(max_depth=10, min_samples_split=5, random_state=42)
dt_model.fit(X_train, y_train)
```

14. Model Evaluation

Evaluate the trained model on both training and validation sets:

Predictions are generated for the sets.

A classification report shows precision, recall, and F1-score.

This allows us to check whether the model generalizes well or is overfitting.

```
y_pred_train = dt_model.predict(X_train)
y_pred_val = dt_model.predict(X_val)

label2id = {label: idx for idx, label in enumerate(sorted(np.unique(labels)))}
id2label = {idx: label for label, idx in label2id.items()}
target_names = [str(id2label[i]) for i in sorted(id2label.keys())]

print("Classification Report (Decison Tree):")
print(classification_report(y_val, y_pred_val, target_names=target_names))

train_accuracy = accuracy_score(y_train, y_pred_train)
val_accuracy = accuracy_score(y_val, y_pred_val)

print(f"Accuracy: {val_accuracy:.4f}")
```

15. Confusion Matrix

The confusion matrix was plotted to visualize class-wise performance**:

Rows = actual labels.

Columns = predicted labels.

Diagonal values = correctly classified samples.

The heatmap helps identify which classes are commonly misclassified.

```
cm = confusion_matrix(y_val, y_pred_val)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=target_names, yticklabels=target_names)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix (Decision Tree)")
plt.show()

from sklearn.preprocessing import label_binarize
from sklearn.metrics import roc_curve, auc
from itertools import cycle
import matplotlib.pyplot as plt

# Binarize labels (One-vs-Rest)
y_val_bin = label_binarize(y_val, classes=sorted(np.unique(labels)))
n_classes = y_val_bin.shape[1]

# Get predicted probabilities
y_pred_proba = dt_model.predict_proba(X_val)

# Compute ROC curve and AUC for each class
fpr, tpr, roc_auc = dict(), dict(), dict()
for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_val_bin[:, i], y_pred_proba[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

# Compute micro-average ROC
fpr["micro"], tpr["micro"], _ = roc_curve(y_val_bin.ravel(), y_pred_proba.ravel())
roc_auc["micro"] = auc(fpr["micro"], tpr["micro"])
```

```
# Plot ROC curves
plt.figure(figsize=(10, 8))
colors = cycle(["blue", "red", "green", "orange", "purple", "brown"])

for i, color in zip(range(n_classes), colors):
    plt.plot(fpr[i], tpr[i], color=color, lw=2,
             label=f"Class {id2label[i]} (AUC = {roc_auc[i]:.2f})")

# Micro-average ROC
plt.plot(fpr["micro"], tpr["micro"], color="black", lw=2, linestyle="--",
         label=f"Micro-average (AUC = {roc_auc['micro']:.2f})")

# Random guess line
plt.plot([0, 1], [0, 1], "k--", lw=1)

plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Decision Tree Classifier")
plt.legend(loc="lower right")
plt.show()
```

16. Save Model and Vocabulary

The model and vocabulary using pickle:

- The trained Decision Tree model is saved.
- The vocabulary dictionary is saved.

This allows reusing the model later for inference without retraining.

```
with open("/Users/ndubuisiobioha/Desktop/Documents/Dissertation/Dissertation
Code/models/decision_tree_cybersecurity.pkl", 'wb') as f:
```

```
    pickle.dump(dt_model, f)
with open("/Users/ndubuisiobioha/Desktop/Documents/Dissertation/Dissertation
Code/models/vocab.pkl", 'wb') as f:
    pickle.dump(vocab, f)
print("Model and vocabulary saved successfully! ✅")
```

Naive Bayes Model Training

Baseline Model: Naive Bayes with spaCy Embeddings

Before applying deep learning with RoBERTa, a baseline was established using Naive Bayes with spaCy word embeddings.

This provides a classical benchmark for text classification, allowing us to compare performance with more advanced models later.

```
# Imports and setup
import pandas as pd
import numpy as np
import spacy
from collections import Counter
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix,
roc_auc_score
from sklearn.preprocessing import label_binarize
%matplotlib inline

import matplotlib.pyplot as plt
import seaborn as sns
import pickle
import sys
import warnings

# Suppress warnings
```

```
warnings.filterwarnings("ignore")

# Check environment
print("Python Version:", sys.version)
print("NumPy Version:", np.__version__)
```

Load Dataset and spaCy Model

the preprocessed cybersecurity news dataset was used and tokenize text using spaCy's medium English model.

```
!python -m spacy download en_core_web_md

# Load spaCy model
nlp = spacy.load("en_core_web_md")

# Load dataset
file_path = "/Users/ndubuisiobioha/Desktop/Documents/Dissertation/Dissertation
Code/Preprocessed/preprocessed_cybersecurity_news.csv"
try:
    df = pd.read_csv(file_path)
except FileNotFoundError:
    print(f"ERROR: File not found at {file_path}")
    sys.exit(1)

# Tokenization
def tokenize_text(text):
    doc = nlp(str(text).lower(), disable=["parser", "ner"])
    return [token.text for token in doc if not token.is_punct and not token.is_space]

df['tokens'] = df['cleaned_text'].apply(tokenize_text)
```

Build Vocabulary and Sequences

We create a vocabulary limited to the 20,000 most common tokens and map each token to an index.
Sequences are padded to a maximum length of 512 tokens.

```
# Build vocabulary
max_vocab_size = 20000
all_tokens = [token for tokens in df['tokens'] for token in tokens]
vocab = Counter(all_tokens)
vocab = {word: idx + 1 for idx, (word, _) in enumerate(vocab.most_common(max_vocab_size))}
vocab['<PAD>'] = 0

# Convert tokens to indices
max_len = 512
def tokens_to_indices(tokens, vocab, max_len=max_len):
    indices = [vocab.get(token, 0) for token in tokens][:max_len]
    return indices

df['indices'] = df['tokens'].apply(lambda x: tokens_to_indices(x, vocab))

# Padding
def custom_pad_sequences(sequences, maxlen, padding='post', value=0):
    padded = np.full((len(sequences), maxlen), value, dtype=np.int32)
    for i, seq in enumerate(sequences):
        seq_len = min(len(seq), maxlen)
        if padding == 'post':
            padded[i, :seq_len] = seq[:seq_len]
        else:
            padded[i, -seq_len:] = seq[:seq_len]
    return padded

sequences = custom_pad_sequences(df['indices'].tolist(), maxlen=max_len, padding='post',
value=0)
labels = df['label_encoded'].values
```

Create Embedding Matrix and Feature Representation

An embedding matrix was built using spaCy's pre-trained vectors.

Each news article is represented by the average embedding of its tokens.

```
# Validate labels
print("Label value counts:", pd.Series(labels).value_counts())
print("Min label:", labels.min(), "Max label:", labels.max())
label_map = {label: idx for idx, label in enumerate(sorted(np.unique(labels)))}
labels = np.array([label_map[label] for label in labels], dtype=np.int32)
print("Remapped label value counts:", pd.Series(labels).value_counts())

# Create embedding matrix
def create_embedding_matrix(vocab, nlp, embedding_dim=300):
    embedding_matrix = np.zeros((len(vocab) + 1, embedding_dim))  # +1 for padding token
    for i, word in enumerate(vocab, 1):
        embedding_matrix[i] = nlp(word).vector
    return embedding_matrix

embedding_matrix = create_embedding_matrix(list(vocab.keys()), nlp, embedding_dim=300)
embedding_matrix = np.nan_to_num(embedding_matrix, nan=0.0, posinf=0.0, neginf=0.0)

# Convert sequences to averaged embeddings
def sequence_to_embedding(sequence, embedding_matrix, max_len=512):
    valid_indices = sequence[sequence != 0]  # Exclude padding
    if len(valid_indices) == 0:
        return np.zeros(embedding_matrix.shape[1])
    embeddings = embedding_matrix[valid_indices]
    return np.mean(embeddings, axis=0)

X = np.array([sequence_to_embedding(seq, embedding_matrix, max_len) for seq in
sequences])
y = labels
```

```
print("Features shape:", X.shape)
print("Labels shape:", y.shape)
print("Vocabulary size:", len(vocab) + 1)
```

Train Naive Bayes Classifier

training of Gaussian Naive Bayes classifier on the averaged embeddings and evaluate on a validation set.

```
from sklearn.model_selection import train_test_split

# Split dataset
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42)

# Initialize and train Naive Bayes
nb_model = GaussianNB()
nb_model.fit(X_train, y_train)

# Predictions
y_pred_train = nb_model.predict(X_train)
y_pred_val = nb_model.predict(X_val)
```

Evaluation Results

The model was asssesed with:

Classification Report

AUC-ROC Score (macro-averaged)

Confusion Matrix Visualization

```
# Reports
label2id = {label: idx for idx, label in enumerate(sorted(np.unique(labels)))}
id2label = {idx: label for label, idx in label2id.items()}
target_names = [str(id2label[i]) for i in sorted(id2label.keys())]

print("Classification Report (Naive Bayes):")
```

```
print(classification_report(y_val, y_pred_val, target_names=target_names))

# Accuracy
train_accuracy = accuracy_score(y_train, y_pred_train)
val_accuracy = accuracy_score(y_val, y_pred_val)
print(f"Accuracy: {val_accuracy:.4f}")

# Confusion Matrix
cm = confusion_matrix(y_val, y_pred_val)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=target_names, yticklabels=target_names)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix (Naive Bayes)")
plt.show()

y_val_bin = label_binarize(y_val, classes=sorted(np.unique(labels)))
n_classes = y_val_bin.shape[1]

# Probabilities
y_pred_proba = nb_model.predict_proba(X_val)

# Compute ROC for each class
fpr, tpr, roc_auc = dict(), dict(), dict()
for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_val_bin[:, i], y_pred_proba[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

# Micro-average ROC
fpr["micro"], tpr["micro"], _ = roc_curve(y_val_bin.ravel(), y_pred_proba.ravel())
roc_auc["micro"] = auc(fpr["micro"], tpr["micro"])
```

```
# Plot
plt.figure(figsize=(10, 8))
colors = cycle(["blue", "red", "green", "orange", "purple", "brown"])
for i, color in zip(range(n_classes), colors):
    plt.plot(fpr[i], tpr[i], color=color, lw=2,
             label=f"Class {id2label[i]} (AUC = {roc_auc[i]:.2f})")

plt.plot(fpr["micro"], tpr["micro"], color="black", lw=2, linestyle="--",
         label=f"Micro-average (AUC = {roc_auc['micro']:.2f})")

plt.plot([0, 1], [0, 1], "k--", lw=1)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Naive Bayes Classifier")
plt.legend(loc="lower right")
plt.show()
```

Save Model and Vocabulary

The trained Naive Bayes model and vocabulary was saved for reuse during inference.

```
# Save model and vocabulary
with open("/Users/ndubuisiobioha/Desktop/Documents/Dissertation/Dissertation
Code/models/naive_bayes_cybersecurity.pkl", 'wb') as f:
    pickle.dump(nb_model, f)
with open("/Users/ndubuisiobioha/Desktop/Documents/Dissertation/Dissertation
Code/models/vocab.pkl", 'wb') as f:
    pickle.dump(vocab, f)
print("Model and vocabulary saved successfully!")
```

RoBERTa Model Training

This notebook covers the model training phase for classifying cybersecurity news article into four threat categories: Data Breach – Vulnerability – Malware – Cyber Attack

We will:
Load the preprocessed dataset
Prepare data for training
Fine-tune RoBERTa for classification
Evaluate the model
Save the trained model

Step 1: Import Required Libraries
We import the necessary libraries for training and evaluating the model

```
# Import fix (must come before all other imports)
import sys
from unittest.mock import MagicMock
sys.modules['transformers.utils.import_utils.importlib_metadata'] = MagicMock()

# Basic Libraries
import pandas as pd
import numpy as np
import torch
from torch.utils.data import Dataset, DataLoader
import torch.nn as nn
import torch.optim as optim
from torch.optim import AdamW

# Hugging Face Transformers (now safe to import)
from transformers import RobertaTokenizer, RobertaForSequenceClassification

# Sklearn for Evaluation
from sklearn.metrics import classification_report, accuracy_score

# Ignore Warnings
```

```
import warnings
warnings.filterwarnings("ignore")

# Check if GPU is available
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)
```

Step 2: Load the Preprocessed Dataset

We load the preprocessed dataset and prepare it for training

```
#Load the preprocessed dataset
file_path =
r"C:\Users\USER\Machine_Learning_CyberThreats\data\preprocessed\preprocessed_cybersec
urity_news.csv"
df = pd.read_csv(file_path)

# Display first few rows
df.head()
```

Step 3: Convert Data into PyTorch Dataset Format

We need to create a custom PyTorch dataset for handling our tokenized inputs

```
class CybersecurityDataset(Dataset):
    def __init__(self, data):
        self.input_ids = data["input_ids"].apply(eval).tolist()
        self.labels = data["label_encoded"].tolist()

    def __len__(self):
        return len(self.input_ids)

    def __getitem__(self, idx):
        return {
            "input_ids": torch.tensor(self.input_ids[idx], dtype=torch.long),
            "label": torch.tensor(self.labels[idx], dtype=torch.long)
```

```
    }

# Create dataset instance
dataset = CybersecurityDataset(df)

# Define DataLoader
batch_size = 8
train_loader = DataLoader(dataset, batch_size=batch_size, shuffle=True)

print("DataLoader is ready for training!")
```

Step 4: Define RoBERTa Model for Classification

We use RobertaForSequenceClassification, which includes a classification head

```
# Load RoBERTa model for classification
num_labels = df['label_encoded'].nunique()  # Get number of unique categories
model = RobertaForSequenceClassification.from_pretrained("roberta-base",
num_labels=num_labels)
model.to(device)
```

Step 5: Define Loss Function & Optimizer

We set up AdamW optimizer and cross-entropy loss for training

```
# Define Loss and Optimizer
criterion = nn.CrossEntropyLoss()
optimizer = AdamW(model.parameters(), lr=2e-5)

# Number of training epochs
num_epochs = 10
```

Step 6: Train the Model

We train the model using PyTorch’s training loop

```
# Training Loop
model.train()
```

```
for epoch in range(num_epochs):
    total_loss = 0
    correct_predictions = 0
    total_samples = 0

    for batch in train_loader:
        optimizer.zero_grad()

        input_ids = batch["input_ids"].to(device)
        labels = batch["label"].to(device)

        # Generate attention mask (1s for real tokens, 0s for padding)
        attention_mask = (input_ids != 0).long()

        # Forward pass with attention mask
        outputs = model(input_ids, attention_mask=attention_mask, labels=labels)
        loss = outputs.loss
        logits = outputs.logits

        loss.backward()
        optimizer.step()

        total_loss += loss.item()
        preds = torch.argmax(logits, dim=1)
        correct_predictions += (preds == labels).sum().item()
        total_samples += labels.size(0)

    avg_loss = total_loss / len(train_loader)
    accuracy = correct_predictions / total_samples

    print(f"Epoch {epoch+1}/{num_epochs}, Loss: {avg_loss:.4f}, Accuracy: {accuracy:.4f}")
```

```
print("Training Completed!")
```

Step 7: Evaluate the Model

We evaluate the trained model on the same dataset since we are not spitting it.

```
# Create label ↔ id mapping
label2id = {label: idx for idx, label in enumerate(sorted(df["label_encoded"].unique()))}
id2label = {idx: label for label, idx in label2id.items()}

# Switch model to evaluation mode
model.eval()

true_labels = []
pred_labels = []

with torch.no_grad():
    for batch in train_loader:
        input_ids = batch["input_ids"].to(device)
        labels = batch["label"].to(device)

        attention_mask = (input_ids != 0).long()

        outputs = model(input_ids, attention_mask=attention_mask)
        logits = outputs.logits
        preds = torch.argmax(logits, dim=1)

        true_labels.extend(labels.cpu().numpy())
        pred_labels.extend(preds.cpu().numpy())

# Classification Report
from sklearn.metrics import classification_report, accuracy_score

# Ensure the target names are strings
target_names = [str(id2label[i]) for i in sorted(id2label.keys())]
```

```
print(classification_report(
    true_labels,
    pred_labels,
    target_names=target_names
))

# Final Accuracy
accuracy = accuracy_score(true_labels, pred_labels)
print(f"Final Model Accuracy: {accuracy:.4f}")
```

AUC ROC

```
# Unset MPLBACKEND to avoid invalid backend error
import os
os.environ.pop('MPLBACKEND', None)  # Remove MPLBACKEND if it exists

# Import required libraries
from sklearn.preprocessing import label_binarize
from sklearn.metrics import roc_curve, roc_auc_score
import numpy as np
import matplotlib.pyplot as plt
import torch

# Explicitly set a valid Matplotlib backend
plt.switch_backend('Agg')  # Use 'Agg' for non-interactive plotting, or 'QtAgg' for interactive

# Assuming label2id, id2label, true_labels, pred_labels are already defined from your working
code
# Create label ↔ id mapping (already in your working code)
label2id = {label: idx for idx, label in enumerate(sorted(df["label_encoded"].unique()))}
id2label = {idx: label for label, idx in label2id.items()}

# Switch model to evaluation mode
```

```
model.eval()

true_labels = []
pred_labels = []
pred_probs = []  # To store prediction probabilities

with torch.no_grad():
    for batch in train_loader:
        input_ids = batch["input_ids"].to(device)
        labels = batch["label"].to(device)

        attention_mask = (input_ids != 0).long()

        outputs = model(input_ids, attention_mask=attention_mask)
        logits = outputs.logits
        probs = torch.softmax(logits, dim=1)  # Convert logits to probabilities
        preds = torch.argmax(logits, dim=1)

        true_labels.extend(labels.cpu().numpy())
        pred_labels.extend(preds.cpu().numpy())
        pred_probs.extend(probs.cpu().numpy())  # Store probabilities

# Convert pred_probs to numpy array
pred_probs = np.array(pred_probs)

# Prepare data for ROC curve
n_classes = len(label2id)
y_true_bin = label_binarize(true_labels, classes=list(range(n_classes))) if n_classes > 2 else
np.array(true_labels)

# Initialize Plotly
plt.figure(figsize=(8, 6))
```

```
# Colors for different classes
colors = ['blue', 'red', 'green', 'orange', 'purple']  # Adjust based on number of classes

# Calculate and plot ROC curve
if n_classes == 2:
    # Binary classification
    fpr, tpr, _ = roc_curve(y_true_bin, pred_probs[:, 1])  # Use probability of positive class
    auc = roc_auc_score(y_true_bin, pred_probs[:, 1])
    plt.plot(fpr, tpr, color=colors[0], label=f'ROC curve (AUC = {auc:.2f})')
else:
    # Multi-class classification (One-vs-Rest)
    for i in range(n_classes):
        fpr, tpr, _ = roc_curve(y_true_bin[:, i], pred_probs[:, i])
        auc = roc_auc_score(y_true_bin[:, i], pred_probs[:, i])
        plt.plot(fpr, tpr, color=colors[i % len(colors)], label=f'Class {id2label[i]} (AUC = {auc:.2f})')

    # Macro-average ROC curve
    all_fpr = np.unique(np.concatenate([roc_curve(y_true_bin[:, i], pred_probs[:, i])[0] for i in
range(n_classes)]))
    mean_tpr = np.zeros_like(all_fpr)
    for i in range(n_classes):
        mean_tpr += np.interp(all_fpr, roc_curve(y_true_bin[:, i], pred_probs[:, i])[0],
roc_curve(y_true_bin[:, i], pred_probs[:, i])[1])
    mean_tpr /= n_classes
    auc_macro = roc_auc_score(y_true_bin, pred_probs, average="macro")
    plt.plot(all_fpr, mean_tpr, color='black', linestyle='--', label=f'Macro-average (AUC =
{auc_macro:.2f})')

# Plot chance line
plt.plot([0, 1], [0, 1], color='gray', linestyle='--', label='Chance')

# Customize plot
plt.xlim([0.0, 1.0])
```

```
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve - RoBERTa Model')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()

# Unset MPLBACKEND to avoid any Matplotlib interference
import os
os.environ.pop('MPLBACKEND', None)

# Import required libraries
from sklearn.preprocessing import label_binarize
from sklearn.metrics import roc_curve, roc_auc_score
import numpy as np
import plotly.graph_objects as go
import plotly.io as pio
import torch

# Set Plotly renderer (adjust based on your environment)
pio.renderers.default = 'jupyterlab'  # Use 'browser' for terminal, 'vscode' for VS Code, or
'notebook' for classic Jupyter

# Assuming label2id, id2label, true_labels, pred_labels are already defined
label2id = {label: idx for idx, label in enumerate(sorted(df["label_encoded"].unique()))}
id2label = {idx: label for label, idx in label2id.items()}

# Switch model to evaluation mode
model.eval()

true_labels = []
pred_labels = []
```

```
pred_probs = []

with torch.no_grad():
    for batch in train_loader:
        input_ids = batch["input_ids"].to(device)
        labels = batch["label"].to(device)
        attention_mask = (input_ids != 0).long()
        outputs = model(input_ids, attention_mask=attention_mask)
        logits = outputs.logits
        probs = torch.softmax(logits, dim=1)
        preds = torch.argmax(logits, dim=1)
        true_labels.extend(labels.cpu().numpy())
        pred_labels.extend(preds.cpu().numpy())
        pred_probs.extend(probs.cpu().numpy())

# Convert pred_probs to numpy array
pred_probs = np.array(pred_probs)

# Prepare data for ROC curve
n_classes = len(label2id)
y_true_bin = label_binarize(true_labels, classes=list(range(n_classes))) if n_classes > 2 else
np.array(true_labels)

# Initialize Plotly figure
fig = go.Figure()

# Colors for different classes
colors = ['#1f77b4', '#ff7f0e', '#2ca02c', '#d62728', '#9467bd']

# Calculate and plot ROC curve
if n_classes == 2:
    fpr, tpr, _ = roc_curve(y_true_bin, pred_probs[:, 1])
    auc = roc_auc_score(y_true_bin, pred_probs[:, 1])
```

```
    fig.add_trace(go.Scatter(
        x=fpr, y=tpr,
        mode='lines',
        name=f'ROC curve (AUC = {auc:.2f})',
        line=dict(color=colors[0], width=2)
    ))
else:
    for i in range(n_classes):
        fpr, tpr, _ = roc_curve(y_true_bin[:, i], pred_probs[:, i])
        auc = roc_auc_score(y_true_bin[:, i], pred_probs[:, i])
        fig.add_trace(go.Scatter(
            x=fpr, y=tpr,
            mode='lines',
            name=f'Class {id2label[i]} (AUC = {auc:.2f})',
            line=dict(color=colors[i % len(colors)], width=2)
        ))

    # Macro-average ROC curve
    all_fpr = np.unique(np.concatenate([roc_curve(y_true_bin[:, i], pred_probs[:, i])[0] for i in
range(n_classes)]))
    mean_tpr = np.zeros_like(all_fpr)
    for i in range(n_classes):
        mean_tpr += np.interp(all_fpr, roc_curve(y_true_bin[:, i], pred_probs[:, i])[0],
roc_curve(y_true_bin[:, i], pred_probs[:, i])[1])
    mean_tpr /= n_classes
    auc_macro = roc_auc_score(y_true_bin, pred_probs, average="macro")
    fig.add_trace(go.Scatter(
        x=all_fpr, y=mean_tpr,
        mode='lines',
        name=f'Macro-average (AUC = {auc_macro:.2f})',
        line=dict(color='black', dash='dash', width=2)
    ))
```

```
# Plot chance line
fig.add_trace(go.Scatter(
    x=[0, 1], y=[0, 1],
    mode='lines',
    name='Chance',
    line=dict(color='gray', dash='dash', width=1.5)
))

# Customize layout
fig.update_layout(
    title='Receiver Operating Characteristic (ROC) Curve - RoBERTa Model',
    xaxis_title='False Positive Rate',
    yaxis_title='True Positive Rate',
    xaxis=dict(range=[0.0, 1.0]),
    yaxis=dict(range=[0.0, 1.05]),
    legend=dict(x=0.7, y=0.1),
    showlegend=True,
    width=800,
    height=600,
    template='plotly_white'
)

# Save or display the plot
try:
    fig.write_image('roc_curve.png')
    print("Plot saved as 'roc_curve.png'")
except Exception as e:
    print(f"Error saving plot: {e}")
    print("Falling back to HTML output...")
    fig.write_html('roc_curve.html')
    print("Plot saved as 'roc_curve.html' (open in a browser)")

try:
```

```
    fig.show()
except Exception as e:
    print(f"Error displaying plot: {e}")
    print("Please check 'roc_curve.html' or ensure your environment supports interactive
display.")
```

```
!pip install --upgrade kaleido
```

Step 8: Save the Trained Model

```
from transformers import RobertaTokenizer

tokenizer = RobertaTokenizer.from_pretrained('roberta-base')

# Save model and tokenizer
model.save_pretrained(r"C:\Users\USER\Machine_Learning_CyberThreats\models\10_roberta_
cybersecurity (3epcoch)")
tokenizer.save_pretrained(r"C:\Users\USER\Machine_Learning_CyberThreats\models\10_robert
a_cybersecurity (3epcoch)")

print("Model saved successfully! ✅")
```

```
import transformers
print(transformers.__version__)
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

# Compute confusion matrix
cm = confusion_matrix(true_labels, pred_labels)

# Plot
plt.figure(figsize=(8, 6))
```

```
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=[id2label[i] for i in sorted(id2label.keys())],
            yticklabels=[id2label[i] for i in sorted(id2label.keys())])
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

import numpy as np
print(np.__version__)
print(np.array([1, 2, 3]))
```